

# Hướng dẫn chuyên sâu về Xử lý ngoại lệ (Exception Handling) trong Spring Boot

---

Xử lý ngoại lệ là một khía cạnh quan trọng trong việc xây dựng các ứng dụng mạnh mẽ và đáng tin cậy. Trong Spring Boot, có nhiều cách tiếp cận để quản lý và xử lý các lỗi xảy ra trong quá trình thực thi, từ các lỗi cơ bản trong code đến các lỗi phức tạp liên quan đến API và cơ sở dữ liệu. Tài liệu này sẽ đi sâu vào tất cả các khía cạnh của xử lý ngoại lệ trong Spring Boot, bao gồm các phương pháp truyền thống, các tính năng do Spring cung cấp, cách tạo ngoại lệ tùy chỉnh, và các best practices.

## Mục lục

---

- Giới thiệu về Ngoại lệ trong Java 1.1. Ngoại lệ là gì? 1.2. Phân loại Ngoại lệ (Checked vs Unchecked) 1.3. Cơ chế `try-catch-finally` 1.4. Từ khóa `throws`
- Xử lý Ngoại lệ trong Spring Boot 2.1. `@ExceptionHandler` (Xử lý ngoại lệ cục bộ) 2.2. `@ControllerAdvice` / `@RestControllerAdvice` (Xử lý ngoại lệ toàn cục) 2.3. `@ResponseStatus` 2.4. `ResponseEntityExceptionHandler` 2.5. `ExceptionHandler` 2.6. `HandlerExceptionResolver`
- Tạo và sử dụng Ngoại lệ tùy chỉnh (Custom Exceptions) 3.1. Khi nào nên tạo Custom Exception? 3.2. Cách tạo Custom Exception 3.3. Sử dụng Custom Exception trong ứng dụng
- Xử lý lỗi Validation 4.1. Bean Validation (JSR 380) 4.2. Xử lý `MethodArgumentNotValidException` 4.3. Xử lý `ConstraintViolationException`
- Xử lý lỗi trong các lớp Service và Repository 5.1. Ngoại lệ trong tầng Service 5.2. Ngoại lệ trong tầng Repository (Spring Data JPA)
- Xử lý lỗi Asynchronous và Reactive 6.1. Xử lý ngoại lệ trong `@Async` methods 6.2. Xử lý ngoại lệ trong Spring WebFlux (Reactive Programming)
- Cấu hình và Tùy chỉnh Error Responses 7.1. Cấu hình `application.properties` cho lỗi 7.2. Tùy chỉnh JSON Error Response 7.3. Sử dụng `ProblemDetail` (Spring 6+)

8. Best Practices và Lời khuyên
- 8.1. Không nuốt ngoại lệ (Don't swallow exceptions)
  - 8.2. Log ngoại lệ đúng cách
  - 8.3. Tạo thông báo lỗi thân thiện với người dùng
  - 8.4. Phân biệt lỗi nghiệp vụ và lỗi hệ thống
  - 8.5. Sử dụng mã lỗi (Error Codes)
  - 8.6. Kiểm thử xử lý ngoại lệ
- 

## 1. Giới thiệu về Ngoại lệ trong Java

---

### 1.1. Ngoại lệ là gì?

Trong lập trình, một **ngoại lệ (exception)** là một sự kiện xảy ra trong quá trình thực thi chương trình, làm gián đoạn luồng bình thường của các lệnh. Khi một ngoại lệ xảy ra, chương trình sẽ tạo ra một đối tượng ngoại lệ (exception object) và

chuyển nó cho hệ thống runtime. Nếu không được xử lý, chương trình sẽ kết thúc đột ngột.

Ngoại lệ là một cơ chế để xử lý các tình huống bất thường một cách có kiểm soát, thay vì để chương trình bị crash.

### 1.2. Phân loại Ngoại lệ (Checked vs Unchecked)

Trong Java, các ngoại lệ được phân thành hai loại chính:

- **Checked Exceptions (Ngoại lệ được kiểm tra):**
  - Là các ngoại lệ mà trình biên dịch yêu cầu phải được xử lý (bằng `try-catch`) hoặc khai báo (`throws`) trong chữ ký phương thức. Nếu không, code sẽ không biên dịch được.
  - Thường đại diện cho các tình huống mà ứng dụng có thể phục hồi được, hoặc các tình huống mà lập trình viên nên biết và xử lý.
  - Ví dụ: `IOException`, `SQLException`, `FileNotFoundException`.
- **Unchecked Exceptions (Ngoại lệ không được kiểm tra):**
  - Là các ngoại lệ không bắt buộc phải được xử lý hoặc khai báo. Chúng thường là các lỗi lập trình (bug) hoặc các tình huống không thể phục hồi được trong quá trình runtime.

- Thường kế thừa từ `RuntimeException` hoặc `Error`.
- Ví dụ: `NullPointerException`, `ArrayIndexOutOfBoundsException`, `IllegalArgumentException`, `ArithmeticException`.
- **Errors:** Là một loại `Throwable` khác, đại diện cho các vấn đề nghiêm trọng mà ứng dụng không thể phục hồi được (ví dụ: `OutOfMemoryError`, `StackOverflowError`). Chúng không nên được `catch`.

### 1.3. Cơ chế `try-catch-finally`

Đây là cơ chế cơ bản để xử lý ngoại lệ trong Java:

- **`try block`:** Chứa đoạn code có khả năng gây ra ngoại lệ.
- **`catch block`:** Chứa đoạn code sẽ được thực thi nếu một ngoại lệ cụ thể xảy ra trong `try block`. Bạn có thể có nhiều `catch block` để xử lý các loại ngoại lệ khác nhau.
- **`finally block`:** Chứa đoạn code luôn được thực thi, bất kể có ngoại lệ xảy ra hay không. Thường dùng để giải phóng tài nguyên (đóng file, kết nối database).

```
public class ExceptionExample {
    public static void main(String[] args) {
        try {
            int result = 10 / 0; // Gây ra ArithmeticException
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.err.println("Lỗi chia cho 0: " + e.getMessage());
        } catch (Exception e) {
            System.err.println("Một lỗi khác đã xảy ra: " + e.getMessage());
        } finally {
            System.out.println("Khôi finally luôn được thực thi.");
        }

        try {
            String text = null;
            System.out.println(text.length()); // Gây ra NullPointerException
        } catch (NullPointerException e) {
            System.err.println("Lỗi Null Pointer: " + e.getMessage());
        }
    }
}
```

## 1.4. Từ khóa throws

Từ khóa `throws` được sử dụng trong chữ ký phương thức để khai báo rằng phương thức đó có thể ném ra một hoặc nhiều loại ngoại lệ được kiểm tra (checked exceptions). Điều này buộc phương thức gọi phải xử lý hoặc khai báo lại ngoại lệ đó.

```
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.IOException;

public class ThrowsExample {

    public void readFile(String filePath) throws FileNotFoundException,
    IOException {
        FileInputStream fis = null;
        try {
            fis = new FileInputStream(filePath);
            int k;
            while ((k = fis.read()) != -1) {
                System.out.print((char) k);
            }
        } finally {
            if (fis != null) {
                fis.close();
            }
        }
    }

    public static void main(String[] args) {
        ThrowsExample example = new ThrowsExample();
        try {
            example.readFile("nonexistent.txt");
        } catch (FileNotFoundException e) {
            System.err.println("Lỗi: Không tìm thấy file - " + e.getMessage());
        } catch (IOException e) {
            System.err.println("Lỗi I/O: " + e.getMessage());
        }
    }
}
```

## 2. Xử lý Ngoại lệ trong Spring Boot

Spring Boot, dựa trên Spring Framework, cung cấp nhiều cơ chế mạnh mẽ và linh hoạt để xử lý ngoại lệ, đặc biệt là trong các ứng dụng web RESTful. Mục tiêu là tập trung logic xử lý lỗi vào một nơi, tránh lặp lại code và cung cấp phản hồi lỗi nhất quán cho client.

## 2.1. @ExceptionHandler (Xử lý ngoại lệ cục bộ)

Annotation `@ExceptionHandler` được sử dụng trong một `@Controller` hoặc `@RestController` để xử lý các ngoại lệ cụ thể được ném ra bởi các phương thức xử lý request trong cùng controller đó. Nó cung cấp một cách để tùy chỉnh phản hồi lỗi cho client khi một ngoại lệ xảy ra.

### Cách sử dụng:

- Tạo một phương thức trong controller.
- Đánh dấu phương thức đó bằng `@ExceptionHandler` và chỉ định loại ngoại lệ mà nó sẽ xử lý.
- Phương thức này có thể trả về `ModelAndView`, `String` (tên view), hoặc `ResponseEntity`.

### Ví dụ:

Giả sử chúng ta có một `ProductController` và muốn xử lý `ProductNotFoundException` chỉ trong controller này.

```

package com.example.demo.controller;

import com.example.demo.exception.ProductNotFoundException;
import com.example.demo.model.Product;
import com.example.demo.service.ProductService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/products")
public class ProductController {

    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping("/{id}")
    public ResponseEntity<Product> getProductById(@PathVariable Long id) {
        return productService.getProductById(id)
            .map(ResponseEntity::ok)
            .orElseThrow(() -> new ProductNotFoundException("Product not
found with ID: " + id));
    }

    // Xử lý ProductNotFoundException chỉ trong ProductController này
    @ExceptionHandler(ProductNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorResponse
handleProductNotFoundException(ProductNotFoundException ex) {
        return new ErrorResponse(HttpStatus.NOT_FOUND.value(),
ex.getMessage());
    }

    // Định nghĩa một lớp ErrorResponse đơn giản
    public static class ErrorResponse {
        private int status;
        private String message;

        public ErrorResponse(int status, String message) {
            this.status = status;
            this.message = message;
        }

        public int getStatus() {
            return status;
        }

        public void setStatus(int status) {
            this.status = status;
        }

        public String getMessage() {
            return message;
        }

        public void setMessage(String message) {
            this.message = message;
        }
    }
}

```

```
}  
}
```

### Ưu điểm:

- Dễ dàng xử lý các ngoại lệ cụ thể cho từng controller.
- Kiểm soát chi tiết phản hồi lỗi.

### Nhược điểm:

- Code xử lý lỗi có thể bị lặp lại nếu nhiều controller ném cùng một loại ngoại lệ.
- Không phù hợp cho việc xử lý ngoại lệ toàn cục.

## 2.2. `@ControllerAdvice` / `@RestControllerAdvice` (Xử lý ngoại lệ toàn cục)

`@ControllerAdvice` (hoặc `@RestControllerAdvice` là một chuyên biệt hóa của `@ControllerAdvice` cho RESTful APIs) là một annotation cấp lớp cho phép bạn định nghĩa các phương thức xử lý ngoại lệ toàn cục cho nhiều controller. Nó hoạt động như một interceptor, bắt các ngoại lệ được ném ra từ bất kỳ controller nào trong ứng dụng.

### Cách sử dụng:

- Tạo một lớp mới và đánh dấu nó bằng `@ControllerAdvice` hoặc `@RestControllerAdvice`.
- Trong lớp này, định nghĩa các phương thức xử lý ngoại lệ tương tự như `@ExceptionHandler` trong controller.

### Ví dụ:

```

package com.example.demo.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.ResponseStatus;

@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ProductNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorResponse
handleProductNotFoundException(ProductNotFoundException ex) {
    return new ErrorResponse(HttpStatus.NOT_FOUND.value(),
ex.getMessage());
}

    @ExceptionHandler(IllegalArgumentException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ErrorResponse
handleIllegalArgumentException(IllegalArgumentException ex) {
    return new ErrorResponse(HttpStatus.BAD_REQUEST.value(),
ex.getMessage());
}

    // Xử lý tất cả các ngoại lệ còn lại
    @ExceptionHandler(Exception.class)
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
    public ErrorResponse handleAllUncaughtException(Exception ex) {
        // Log lỗi chi tiết ở đây
        System.err.println("An unexpected error occurred: " + ex.getMessage());
        return new ErrorResponse(HttpStatus.INTERNAL_SERVER_ERROR.value(), "An
unexpected error occurred");
    }

    // Định nghĩa lại ErrorResponse nếu cần, hoặc sử dụng lại từ ví dụ trước
    public static class ErrorResponse {
        private int status;
        private String message;

        public ErrorResponse(int status, String message) {
            this.status = status;
            this.message = message;
        }

        public int getStatus() {
            return status;
        }

        public void setStatus(int status) {
            this.status = status;
        }

        public String getMessage() {
            return message;
        }

        public void setMessage(String message) {
            this.message = message;
        }
    }
}

```



```
}  
}  
}
```

#### Ưu điểm:

- **Tập trung hóa:** Tất cả logic xử lý lỗi được đặt ở một nơi duy nhất, giúp code sạch sẽ và dễ bảo trì.
- **Nhất quán:** Đảm bảo phản hồi lỗi nhất quán trên toàn bộ ứng dụng.
- **Tách biệt mối quan tâm:** Tách biệt logic xử lý lỗi khỏi logic nghiệp vụ của controller.

#### Nhược điểm:

- Có thể trở nên phức tạp nếu có quá nhiều loại ngoại lệ cần xử lý.

### 2.3. @ResponseStatus

Annotation `@ResponseStatus` có thể được sử dụng trực tiếp trên một lớp ngoại lệ tùy chỉnh hoặc trên một phương thức `@ExceptionHandler` để chỉ định mã trạng thái HTTP sẽ được trả về khi ngoại lệ đó xảy ra. Khi được đặt trên một lớp ngoại lệ, Spring sẽ tự động trả về mã trạng thái HTTP được chỉ định khi ngoại lệ đó được ném ra.

#### Ví dụ trên lớp ngoại lệ tùy chỉnh:

```
package com.example.demo.exception;  
  
import org.springframework.http.HttpStatus;  
import org.springframework.web.bind.annotation.ResponseStatus;  
  
@ResponseStatus(HttpStatus.NOT_FOUND)  
public class ProductNotFoundException extends RuntimeException {  
    public ProductNotFoundException(String message) {  
        super(message);  
    }  
}
```

Khi `ProductNotFoundException` được ném ra, Spring sẽ tự động trả về HTTP status code 404 (Not Found). Bạn không cần phải xử lý nó trong `@ExceptionHandler` nếu chỉ muốn thiết lập status code.

#### Ưu điểm:

- Đơn giản, gọn gàng cho các trường hợp chỉ cần thiết lập HTTP status code.

- Tự động hóa việc ánh xạ ngoại lệ với status code.

### Nhược điểm:

- Không cho phép tùy chỉnh body của phản hồi lỗi một cách linh hoạt như `@ExceptionHandler`.
- Chỉ hoạt động với `RuntimeException` hoặc các ngoại lệ được đánh dấu bằng `@ResponseStatus`.

## 2.4. `ResponseEntityExceptionHandler`

`ResponseEntityExceptionHandler` là một lớp tiện ích được Spring cung cấp, là một `ControllerAdvice` cơ bản, xử lý các ngoại lệ Spring MVC nội bộ (ví dụ: `MethodArgumentNotValidException`, `NoHandlerFoundException`, `HttpMessageNotReadableException`). Bằng cách kế thừa lớp này, bạn có thể dễ dàng ghi đè (override) các phương thức xử lý cho các ngoại lệ Spring MVC cụ thể và tùy chỉnh phản hồi lỗi.

### Cách sử dụng:

- Tạo một lớp mới kế thừa `ResponseEntityExceptionHandler` và đánh dấu nó bằng `@ControllerAdvice` hoặc `@RestControllerAdvice`.
- Ghi đè các phương thức `handle...` tương ứng với các ngoại lệ Spring MVC mà bạn muốn tùy chỉnh.

### Ví dụ:

```

package com.example.demo.exception;

import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.HttpStatusCode;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.context.request.WebRequest;
import
org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler

import java.time.LocalDateTime;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

@ControllerAdvice
public class CustomResponseEntityExceptionHandler extends
ResponseEntityExceptionHandler {

    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(
        MethodArgumentNotValidException ex, HttpHeaders headers,
        HttpStatus status, WebRequest request) {

        Map<String, Object> body = new LinkedHashMap<>();
        body.put("timestamp", LocalDateTime.now());
        body.put("status", status.value());

        List<String> errors = ex.getBindingResult()
            .getFieldErrors()
            .stream()
            .map(x -> x.getDefaultMessage())
            .collect(Collectors.toList());

        body.put("errors", errors);

        return new ResponseEntity<>(body, headers, status);
    }

    // Bạn có thể ghi đè các phương thức handle khác ở đây, ví dụ:
    // handleNoHandlerFoundException, handleHttpMessageNotReadable, etc.
}

```

### Ưu điểm:

- Cung cấp một điểm khởi đầu tốt để xử lý các ngoại lệ Spring MVC tiêu chuẩn.
- Giúp tùy chỉnh phản hồi lỗi cho các ngoại lệ phổ biến mà không cần viết lại từ đầu.

### Nhược điểm:

- Chỉ xử lý các ngoại lệ được định nghĩa trong `ResponseEntityExceptionHandler` . Đối với các ngoại lệ tùy chỉnh hoặc ngoại lệ không được Spring MVC xử lý, bạn vẫn cần sử dụng `@ExceptionHandler` riêng.

## 2.5. `ErrorController`

Khi một lỗi xảy ra trong ứng dụng Spring Boot (ví dụ: một request đến một URL không tồn tại, hoặc một ngoại lệ không được xử lý bởi `@ExceptionHandler` nào), Spring Boot sẽ chuyển hướng request đến một path mặc định ( `/error` ). `ErrorController` là một controller được Spring Boot cung cấp để xử lý các request đến path này.

Bạn có thể tùy chỉnh trang lỗi mặc định bằng cách tạo một lớp implement `ErrorController` và ghi đè phương thức `getErrorPath()` (để chỉ định path xử lý lỗi) và phương thức xử lý request cho path đó.

**Ví dụ:**

```

package com.example.demo.controller;

import jakarta.servlet.RequestDispatcher;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.boot.web.servlet.error.ErrorController;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseBody;

import java.util.HashMap;
import java.util.Map;

@Controller
@RequestMapping("/error") // Mặc định là /error
public class CustomErrorController implements ErrorController {

    @RequestMapping
    @ResponseBody
    public ResponseEntity<Map<String, Object>> handleError(HttpServletRequest
request) {
        HttpStatus status = getStatus(request);
        Map<String, Object> errorAttributes = new HashMap<>();
        errorAttributes.put("timestamp", System.currentTimeMillis());
        errorAttributes.put("status", status.value());
        errorAttributes.put("error", status.getReasonPhrase());

        Object message = request.getAttribute(RequestDispatcher.ERROR_MESSAGE);
        if (message != null) {
            errorAttributes.put("message", message);
        } else {
            errorAttributes.put("message", "An error occurred");
        }

        Object path =
request.getAttribute(RequestDispatcher.ERROR_REQUEST_URI);
        if (path != null) {
            errorAttributes.put("path", path);
        }

        return new ResponseEntity<>(errorAttributes, status);
    }

    private HttpStatus getStatus(HttpServletRequest request) {
        Integer statusCode = (Integer)
request.getAttribute(RequestDispatcher.ERROR_STATUS_CODE);
        if (statusCode == null) {
            return HttpStatus.INTERNAL_SERVER_ERROR;
        }
        try {
            return HttpStatus.valueOf(statusCode);
        } catch (IllegalArgumentException ex) {
            return HttpStatus.INTERNAL_SERVER_ERROR;
        }
    }

    // Không cần ghi đè getErrorPath() nếu bạn muốn sử dụng path mặc định
    /error
    // @Override
    // public String getErrorPath() {

```

```
//     return "/error";  
// }  
}
```

### Ưu điểm:

- Xử lý các lỗi không được bắt bởi các cơ chế khác (ví dụ: lỗi 404, 401, 403).
- Cung cấp một fallback mechanism cho tất cả các lỗi.

### Nhược điểm:

- Ít linh hoạt hơn trong việc xử lý các ngoại lệ cụ thể so với `@ControllerAdvice`.
- Thường được sử dụng để hiển thị trang lỗi chung hoặc trả về phản hồi JSON lỗi chung.

## 2.6. `HandlerExceptionResolver`

`HandlerExceptionResolver` là một interface trong Spring Framework cho phép bạn tùy chỉnh cách xử lý ngoại lệ trong Spring MVC. Đây là một cơ chế cấp thấp hơn so với `@ExceptionHandler` và `@ControllerAdvice`, cung cấp quyền kiểm soát chi tiết hơn về quá trình xử lý ngoại lệ.

### Cách sử dụng:

- Tạo một lớp implement `HandlerExceptionResolver`.
- Ghi đè phương thức `resolveException()`.
- Đăng ký bean này trong cấu hình Spring.

### Ví dụ:

```

package com.example.demo.exception;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.stereotype.Component;
import org.springframework.web.servlet.HandlerExceptionResolver;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.view.json.MappingJackson2JsonView;

import java.util.HashMap;
import java.util.Map;

@Component
public class CustomHandlerExceptionResolver implements HandlerExceptionResolver
{
    @Override
    public ModelAndView resolveException(
        HttpServletRequest request, HttpServletResponse response, Object
        handler, Exception ex) {

        ModelAndView modelAndView = new ModelAndView();
        MappingJackson2JsonView jsonView = new MappingJackson2JsonView();
        Map<String, Object> errorAttributes = new HashMap<>();

        if (ex instanceof ProductNotFoundException) {
            response.setStatus(HttpServletResponse.SC_NOT_FOUND);
            errorAttributes.put("status", HttpServletResponse.SC_NOT_FOUND);
            errorAttributes.put("message", ex.getMessage());
        } else if (ex instanceof IllegalArgumentException) {
            response.setStatus(HttpServletResponse.SC_BAD_REQUEST);
            errorAttributes.put("status", HttpServletResponse.SC_BAD_REQUEST);
            errorAttributes.put("message", ex.getMessage());
        } else {
            response.setStatus(HttpServletResponse.SC_INTERNAL_SERVER_ERROR);
            errorAttributes.put("status",
                HttpServletResponse.SC_INTERNAL_SERVER_ERROR);
            errorAttributes.put("message", "An unexpected error occurred");
        }

        jsonView.setAttributesMap(errorAttributes);
        modelAndView.setView(jsonView);
        return modelAndView;
    }
}

```

### Ưu điểm:

- Cung cấp quyền kiểm soát cao nhất đối với quá trình xử lý ngoại lệ.
- Có thể được sử dụng để xử lý các ngoại lệ từ bất kỳ `Handler` nào, không chỉ `controller`.

### Nhược điểm:

- Phức tạp hơn để triển khai so với `@ControllerAdvice`.

- Thường chỉ được sử dụng trong các trường hợp đặc biệt khi các cơ chế khác không đủ linh hoạt.
- 

## 3. Tạo và sử dụng Ngoại lệ tùy chỉnh (Custom Exceptions)

---

Việc tạo các ngoại lệ tùy chỉnh giúp ứng dụng của bạn rõ ràng hơn về các loại lỗi nghiệp vụ cụ thể, cải thiện khả năng đọc code và giúp client hiểu rõ hơn về lý do lỗi.

### 3.1. Khi nào nên tạo Custom Exception?

Bạn nên tạo custom exception khi:

- **Lỗi nghiệp vụ cụ thể:** Khi một tình huống lỗi phản ánh một quy tắc nghiệp vụ bị vi phạm (ví dụ: `InsufficientStockException`, `UserNotFoundException`, `InvalidOrderStateException`).
- **Cần thêm thông tin:** Khi bạn muốn đính kèm thêm dữ liệu vào ngoại lệ (ví dụ: mã lỗi, chi tiết validation).
- **Cần phân biệt rõ ràng:** Khi các ngoại lệ Java tiêu chuẩn không đủ để mô tả chính xác vấn đề.
- **Tăng tính đọc hiểu:** Giúp code dễ đọc và dễ hiểu hơn về các loại lỗi có thể xảy ra.

### 3.2. Cách tạo Custom Exception

Một custom exception thường kế thừa từ `RuntimeException` (để trở thành unchecked exception) hoặc `Exception` (để trở thành checked exception). Đối với các ứng dụng Spring Boot RESTful, thường nên sử dụng `RuntimeException` để tránh việc phải khai báo `throws` ở khắp mọi nơi, và xử lý chúng tập trung bằng `@ControllerAdvice`.

**Ví dụ:** `ResourceNotFoundException`



```

package com.example.demo.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(HttpStatus.NOT_FOUND) // Tự động tra' về` 404 Not Found
public class ResourceNotFoundException extends RuntimeException {

    public ResourceNotFoundException(String message) {
        super(message);
    }

    public ResourceNotFoundException(String message, Throwable cause) {
        super(message, cause);
    }
}

```

Ví dụ: `InvalidInputException` (có thể chứa thêm thông tin)

```

package com.example.demo.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

import java.util.Map;

@ResponseStatus(HttpStatus.BAD_REQUEST) // Tự động tra' về` 400 Bad Request
public class InvalidInputException extends RuntimeException {

    private final Map<String, String> errors;

    public InvalidInputException(String message) {
        super(message);
        this.errors = null;
    }

    public InvalidInputException(String message, Map<String, String> errors) {
        super(message);
        this.errors = errors;
    }

    public Map<String, String> getErrors() {
        return errors;
    }
}

```

### 3.3. Sử dụng Custom Exception trong ứng dụng

Bạn có thể ném các custom exception từ bất kỳ tầng nào trong ứng dụng (controller, service, repository).

Ví dụ trong tầng Service:

```

package com.example.demo.service;

import com.example.demo.exception.ResourceNotFoundException;
import com.example.demo.model.Product;
import com.example.demo.repository.ProductRepository;
import org.springframework.stereotype.Service;

import java.util.Optional;

@Service
public class ProductService {

    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    public Product getProductById(Long id) {
        return productRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Product not
found with ID: " + id));
    }

    public Product updateProduct(Long id, Product updatedProduct) {
        return productRepository.findById(id)
            .map(existingProduct -> {
                existingProduct.setName(updatedProduct.getName());
                existingProduct.setPrice(updatedProduct.getPrice());
                return productRepository.save(existingProduct);
            })
            .orElseThrow(() -> new ResourceNotFoundException("Product not
found for update with ID: " + id));
    }
}

```

**Xử lý Custom Exception trong @ControllerAdvice :**

```

package com.example.demo.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorDetails>
handleResourceNotFoundException(ResourceNotFoundException ex) {
        ErrorDetails errorDetails = new ErrorDetails(
            LocalDateTime.now(), ex.getMessage(), "Resource Not Found");
        return new ResponseEntity<>(errorDetails, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(InvalidInputException.class)
    public ResponseEntity<ErrorDetails>
handleInvalidInputException(InvalidInputException ex) {
        Map<String, Object> details = new HashMap<>();
        details.put("message", ex.getMessage());
        if (ex.getErrors() != null) {
            details.put("validationErrors", ex.getErrors());
        }
        ErrorDetails errorDetails = new ErrorDetails(
            LocalDateTime.now(), "Invalid Input", details);
        return new ResponseEntity<>(errorDetails, HttpStatus.BAD_REQUEST);
    }

    // Lớp ErrorDetails để tra về phần hồ i l i nhâ t quán
    public static class ErrorDetails {
        private LocalDateTime timestamp;
        private String message;
        private Object details;

        public ErrorDetails(LocalDateTime timestamp, String message, Object
details) {
            this.timestamp = timestamp;
            this.message = message;
            this.details = details;
        }

        // Getters and Setters
        public LocalDateTime getTimestamp() {
            return timestamp;
        }

        public void setTimestamp(LocalDateTime timestamp) {
            this.timestamp = timestamp;
        }

        public String getMessage() {
            return message;
        }
    }
}

```

```
    public void setMessage(String message) {
        this.message = message;
    }

    public Object getDetails() {
        return details;
    }

    public void setDetails(Object details) {
        this.details = details;
    }
}
}
```

## 4. Xử lý lỗi Validation

Validation (kiểm tra dữ liệu đầu vào) là một phần quan trọng để đảm bảo tính toàn vẹn và chính xác của dữ liệu trong ứng dụng. Spring Boot tích hợp tốt với Bean Validation (JSR 380) để xử lý validation. Khi validation thất bại, các ngoại lệ cụ thể sẽ được ném ra.

### 4.1. Bean Validation (JSR 380)

Bean Validation là một tiêu chuẩn Java API để kiểm tra ràng buộc dữ liệu. Nó cho phép bạn định nghĩa các ràng buộc (constraints) trên các trường của đối tượng bằng cách sử dụng các annotation. Spring Boot tự động tích hợp Hibernate Validator (một implementation của Bean Validation).

#### Thêm Dependency:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

#### Ví dụ về Bean Validation:

```

package com.example.demo.dto;

import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;

public class ProductDto {

    @NotBlank(message = "Tên sản phẩm không được để trống")
    @Size(min = 3, max = 100, message = "Tên sản phẩm phải từ 3 đến 100 ký tự")
    private String name;

    @NotNull(message = "Giá sản phẩm không được để trống")
    @Min(value = 0, message = "Giá sản phẩm phải lớn hơn hoặc bằng 0")
    private Double price;

    // Constructors, getters, setters
    public ProductDto() {}

    public ProductDto(String name, Double price) {
        this.name = name;
        this.price = price;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public Double getPrice() {
        return price;
    }

    public void setPrice(Double price) {
        this.price = price;
    }
}

```

Để kích hoạt validation trên controller, bạn sử dụng `@Valid` hoặc `@Validated`:

```

package com.example.demo.controller;

import com.example.demo.dto.ProductDto;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/products")
public class ProductController {

    @PostMapping
    public ResponseEntity<String> createProduct(@Valid @RequestBody ProductDto
productDto) {
        // Nếu validation thành công, code ở đây sẽ được thực thi
        return ResponseEntity.ok("Product created: " + productDto.getName());
    }
}

```

## 4.2. Xử lý `MethodArgumentNotValidException`

Khi validation thất bại trên một đối tượng được đánh dấu `@Valid` hoặc `@Validated` trong controller, Spring sẽ ném ra `MethodArgumentNotValidException`. Bạn có thể xử lý ngoại lệ này trong `@ControllerAdvice`.

```

package com.example.demo.exception;

import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.HttpStatusCode;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.context.request.WebRequest;
import
org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

@ControllerAdvice
public class GlobalValidationExceptionHandler extends
ResponseEntityExceptionHandler {

    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(
        MethodArgumentNotValidException ex, HttpHeaders headers,
        HttpStatusCode status, WebRequest request) {

        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getAllErrors().forEach(error -> {
            String fieldName = ((FieldError) error).getField();
            String errorMessage = error.getDefaultMessage();
            errors.put(fieldName, errorMessage);
        });

        Map<String, Object> body = new LinkedHashMap<>();
        body.put("timestamp", LocalDateTime.now());
        body.put("status", status.value());
        body.put("message", "Validation failed");
        body.put("errors", errors);

        return new ResponseEntity<>(body, headers, status);
    }
}

```

Phản hồi JSON sẽ trông như sau:

```

{
  "timestamp": "2023-10-27T10:30:00.123456789",
  "status": 400,
  "message": "Validation failed",
  "errors": {
    "name": "Tên sản phẩm phải từ 3 đến 100 ký tự",
    "price": "Giá sản phẩm không được để trống"
  }
}

```

### 4.3. Xử lý `ConstraintViolationException`

`ConstraintViolationException` được ném ra khi validation thất bại trên các phương thức được đánh dấu `@Validated` (không phải `@Valid` trên request body) hoặc khi validation được kích hoạt thủ công bằng `Validator` API.

Ví dụ:

```
package com.example.demo.service;

import jakarta.validation.ConstraintViolationException;
import jakarta.validation.constraints.Min;
import org.springframework.stereotype.Service;
import org.springframework.validation.annotation.Validated;

@Service
@Validated // Kích hoạt validation cho các phương thức trong service này
public class OrderService {

    public String processOrder(@Min(value = 1, message = "Số lượng phải lớn hơn 0") int quantity) {
        if (quantity > 100) {
            throw new IllegalArgumentException("Số lượng không được vượt quá 100");
        }
        return "Order processed with quantity: " + quantity;
    }
}
```

Xử lý trong `@ControllerAdvice` :



```

package com.example.demo.exception;

import jakarta.validation.ConstraintViolation;
import jakarta.validation.ConstraintViolationException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalValidationExceptionHandler {

    @ExceptionHandler(ConstraintViolationException.class)
    public ResponseEntity<Map<String, Object>>
handleConstraintViolationException(ConstraintViolationException ex) {
        Map<String, String> errors = new HashMap<>();
        for (ConstraintViolation<?> violation : ex.getConstraintViolations()) {
            errors.put(violation.getPropertyPath().toString(),
violation.getMessage());
        }

        Map<String, Object> body = new LinkedHashMap<>();
        body.put("timestamp", LocalDateTime.now());
        body.put("status", HttpStatus.BAD_REQUEST.value());
        body.put("message", "Validation failed");
        body.put("errors", errors);

        return new ResponseEntity<>(body, HttpStatus.BAD_REQUEST);
    }
}

```

## 5. Xử lý lỗi trong các lớp Service và Repository

Việc xử lý ngoại lệ không chỉ giới hạn ở tầng controller. Các tầng Service và Repository cũng cần có chiến lược xử lý lỗi phù hợp để đảm bảo tính toàn vẹn dữ liệu và cung cấp thông tin lỗi có ý nghĩa.

### 5.1. Ngoại lệ trong tầng Service

Tầng Service là nơi chứa logic nghiệp vụ chính của ứng dụng. Khi có lỗi xảy ra ở tầng này, thường là do vi phạm các quy tắc nghiệp vụ hoặc dữ liệu không hợp lệ. Đây là nơi lý tưởng để ném các **custom business exceptions**.

**Ví dụ:**

```

package com.example.demo.service;

import com.example.demo.exception.InsufficientStockException;
import com.example.demo.exception.ResourceNotFoundException;
import com.example.demo.model.Order;
import com.example.demo.model.Product;
import com.example.demo.repository.OrderRepository;
import com.example.demo.repository.ProductRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class OrderService {

    private final OrderRepository orderRepository;
    private final ProductRepository productRepository;

    public OrderService(OrderRepository orderRepository, ProductRepository
productRepository) {
        this.orderRepository = orderRepository;
        this.productRepository = productRepository;
    }

    @Transactional
    public Order createOrder(Long productId, int quantity) {
        Product product = productRepository.findById(productId)
            .orElseThrow(() -> new ResourceNotFoundException("Product not
found with ID: " + productId));

        if (product.getStock() < quantity) {
            throw new InsufficientStockException("Not enough stock for product:
" + product.getName());
        }

        product.setStock(product.getStock() - quantity);
        productRepository.save(product);

        Order order = new Order();
        order.setProductId(productId);
        order.setQuantity(quantity);
        order.setTotalPrice(product.getPrice() * quantity);

        return orderRepository.save(order);
    }
}

```

Trong ví dụ trên, `InsufficientStockException` là một custom exception đại diện cho một lỗi nghiệp vụ cụ thể. Ngoại lệ này sau đó có thể được bắt và xử lý bởi `@ControllerAdvice` để trả về phản hồi lỗi phù hợp cho client (ví dụ: HTTP 400 Bad Request).

## 5.2. Ngoại lệ trong tầng Repository (Spring Data JPA)

Tầng Repository tương tác trực tiếp với cơ sở dữ liệu. Các lỗi ở tầng này thường liên quan đến vấn đề kết nối database, vi phạm ràng buộc dữ liệu (ví dụ: unique constraint), hoặc các vấn đề về truy vấn.

Spring Data JPA thường chuyển đổi các ngoại lệ cấp thấp của JDBC/JPA thành các ngoại lệ cụ thể của Spring Framework (ví dụ: `DataAccessException`). Bạn có thể bắt các ngoại lệ này để xử lý hoặc ném ra các custom exception nghiệp vụ.

**Ví dụ:**

```
package com.example.demo.repository;

import com.example.demo.model.Product;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {

    Optional<Product> findByName(String name);

    default Product saveProduct(Product product) {
        try {
            return save(product);
        } catch (DataIntegrityViolationException ex) {
            // Xử lý lỗi khi có vi phạm ràng buộc duy nhất (unique
            // constraint)
            if (ex.getMessage().contains("ConstraintViolationException")) {
                throw new DuplicateProductException("Product with name '" +
                product.getName() + "' already exists.", ex);
            }
            throw ex; // Ném lại các loại DataIntegrityViolationException khác
        }
    }
}
```

```
// DuplicateProductException.java
package com.example.demo.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(HttpStatus.CONFLICT) // HTTP 409 Conflict
public class DuplicateProductException extends RuntimeException {
    public DuplicateProductException(String message) {
        super(message);
    }

    public DuplicateProductException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

Trong ví dụ này, `DataIntegrityViolationException` được bắt và chuyển đổi thành `DuplicateProductException` (một custom exception) để cung cấp thông tin lỗi rõ ràng hơn cho tầng trên.

---

## 6. Xử lý lỗi Asynchronous và Reactive

---

Khi làm việc với các tác vụ không đồng bộ (asynchronous) hoặc lập trình phản ứng (reactive programming), cơ chế xử lý ngoại lệ có thể khác biệt so với luồng đồng bộ truyền thống.

### 6.1. Xử lý ngoại lệ trong `@Async` methods

Khi một phương thức được đánh dấu `@Async` ném ra một ngoại lệ, ngoại lệ đó sẽ không được ném trực tiếp trở lại luồng gọi. Thay vào đó, nó sẽ được bắt bởi một `AsyncUncaughtExceptionHandler`.

**Cách sử dụng:**

1. **Kích hoạt Async:** Đảm bảo lớp cấu hình của bạn có `@EnableAsync`.
2. **Tạo `AsyncUncaughtExceptionHandler`:** Implement interface này.
3. **Cấu hình `AsyncUncaughtExceptionHandler`:** Ghi đè phương thức `getAsyncUncaughtExceptionHandler()` trong lớp cấu hình `AsyncConfigurer`.

**Ví dụ:**

```

package com.example.demo.config;

import org.springframework.aop.interceptor.AsyncUncaughtExceptionHandler;
import org.springframework.context.annotation.Configuration;
import org.springframework.scheduling.annotation.AsyncConfigurer;
import org.springframework.scheduling.annotation.EnableAsync;
import org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor;

import java.lang.reflect.Method;
import java.util.concurrent.Executor;

@Configuration
@EnableAsync
public class AsyncConfig implements AsyncConfigurer {

    @Override
    public Executor getAsyncExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(2);
        executor.setMaxPoolSize(5);
        executor.setQueueCapacity(10);
        executor.setThreadNamePrefix("MyAsync-");
        executor.initialize();
        return executor;
    }

    @Override
    public AsyncUncaughtExceptionHandler getAsyncUncaughtExceptionHandler() {
        return new CustomAsyncExceptionHandler();
    }

    public static class CustomAsyncExceptionHandler implements
        AsyncUncaughtExceptionHandler {

        @Override
        public void handleUncaughtException(
            Throwable ex, Method method, Object... params) {

            System.err.println("Exception message - " + ex.getMessage());
            System.err.println("Method name - " + method.getName());
            for (Object param : params) {
                System.err.println("Parameter value - " + param);
            }
            // Ghi log lỗi chi tiết, gửi thông báo, v.v.
        }
    }
}

```

Ví dụ về phương thức `@Async` ném ngoại lệ:

```

package com.example.demo.service;

import org.springframework.scheduling.annotation.Async;
import org.springframework.stereotype.Service;

@Service
public class AsyncService {

    @Async
    public void performAsyncTask(String data) {
        System.out.println("Performing async task with data: " + data + " on
thread: " + Thread.currentThread().getName());
        if (data.equals("error")) {
            throw new RuntimeException("Simulated async error for data: " +
data);
        }
        System.out.println("Async task completed for data: " + data);
    }
}

```

Khi bạn gọi `asyncService.performAsyncTask("error")`, ngoại lệ sẽ được bắt bởi `CustomAsyncExceptionHandler`.

## 6.2. Xử lý ngoại lệ trong Spring WebFlux (Reactive Programming)

Trong lập trình reactive với Spring WebFlux, ngoại lệ được xử lý thông qua các operator trên `Mono` và `Flux`. Các ngoại lệ không làm gián đoạn luồng thực thi mà được truyền xuống downstream như một tín hiệu lỗi.

### Các operator xử lý lỗi phổ biến:

- `onErrorResume(Function<? super Throwable, ? extends Publisher<? extends T>> fallback)`: Khi một lỗi xảy ra, nó sẽ chuyển sang một `Publisher` thay thế. Hữu ích cho việc cung cấp giá trị mặc định hoặc thực hiện logic phục hồi.

```

java public Mono<Product> getProductById(String id) { return
productRepository.findById(id) .onErrorResume(e -> { // Log lỗi
System.err.println("Error fetching product: " + e.getMessage()); //
Trả về một Mono rỗng hoặc một đối tượng mặc định return Mono.empty();
// Hoặc: return Mono.just(new Product("Default Product", 0.0)); }); }

```

- `onErrorReturn(T value)`: Khi một lỗi xảy ra, nó sẽ trả về một giá trị mặc định cụ thể.

```
java public Mono<Product> getProductByIdOrDefault(String id) { return
productRepository.findById(id) .onErrorReturn(new Product("Fallback
Product", 0.0)); }
```

- **onErrorMap(Function<? super Throwable, ? extends Throwable> errorHandler) :** Chuyển đổi một loại ngoại lệ này sang một loại ngoại lệ khác. Hữu ích để ánh xạ các ngoại lệ cấp thấp thành các ngoại lệ nghiệp vụ.

```
java public Mono<Product> getProductByIdAndMapError(String id) {
return productRepository.findById(id)
.onErrorMap(DataAccessException.class, e -> new
DatabaseOperationException("Database error", e)); }
```

- **doOnError(Consumer<? super Throwable> onErrorConsumer) :** Thực hiện một hành động phụ (side effect) khi lỗi xảy ra, nhưng không thay đổi luồng lỗi. Thường dùng để ghi log.

```
java public Flux<Product> getAllProductsAndLogErrors() { return
productRepository.findAll() .doOnError(e -> System.err.println("Error
in product stream: " + e.getMessage())); }
```

- **retry(long numRetries) :** Thử lại numRetries lần khi lỗi xảy ra.

```
java public Mono<Product> getProductWithRetry(String id) { return
productRepository.findById(id) .retry(3); // Thử lại 3 lần nếu có lỗi
}
```

- **switchIfEmpty(Mono<? extends T> alternate) :** Nếu Mono rỗng (không có phần tử nào được phát ra), chuyển sang một Mono thay thế. Thường dùng để ném ngoại lệ khi không tìm thấy tài nguyên.

```
java public Mono<Product> getProductOrThrowNotFound(String id) {
return productRepository.findById(id) .switchIfEmpty(Mono.error(new
ResourceNotFoundException("Product not found with ID: " + id))); }
```

## Xử lý lỗi toàn cục trong WebFlux:

Giống như Spring MVC, WebFlux cũng hỗ trợ xử lý lỗi toàn cục thông qua `ErrorWebExceptionHandler` hoặc `@ControllerAdvice`.

- **ErrorWebExceptionHandler** : Interface để xử lý lỗi cấp thấp trong WebFlux. Bạn có thể implement interface này để tùy chỉnh phản hồi lỗi cho toàn bộ ứng dụng.

```
```java package com.example.demo.exception;

import
org.springframework.boot.web.reactive.error.ErrorWebExceptionHandler;
import          org.springframework.core.annotation.Order;          import
org.springframework.core.io.buffer.DataBufferFactory;              import
org.springframework.http.HttpStatus;                                import
org.springframework.http.MediaType;                                import
org.springframework.http.server.reactive.ServerHttpResponse;       import
org.springframework.stereotype.Component;                           import
org.springframework.web.server.ServerWebExchange;                  import
reactor.core.publisher.Mono;

import java.time.LocalDateTime; import java.util.LinkedHashMap; import
java.util.Map;

@Component @Order(-1) // Đảm bảo handler này được gọi trước các handler
khác public class GlobalReactiveExceptionHandler implements
ErrorWebExceptionHandler {
```



```

@Override
public Mono<Void> handle(ServerWebExchange exchange, Throwable ex) {
    ServerHttpResponse response = exchange.getResponse();
    response.getHeaders().setContentType(MediaType.APPLICATION_JSON);

    Map<String, Object> errorAttributes = new LinkedHashMap<>();
    errorAttributes.put("timestamp", LocalDateTime.now().toString());

    if (ex instanceof ResourceNotFoundException) {
        response.setStatusCode(HttpStatus.NOT_FOUND);
        errorAttributes.put("status", HttpStatus.NOT_FOUND.value());
        errorAttributes.put("error", "Not Found");
        errorAttributes.put("message", ex.getMessage());
    } else if (ex instanceof IllegalArgumentException) {
        response.setStatusCode(HttpStatus.BAD_REQUEST);
        errorAttributes.put("status", HttpStatus.BAD_REQUEST.value());
        errorAttributes.put("error", "Bad Request");
        errorAttributes.put("message", ex.getMessage());
    } else {
        response.setStatusCode(HttpStatus.INTERNAL_SERVER_ERROR);
        errorAttributes.put("status",
HttpStatus.INTERNAL_SERVER_ERROR.value());
        errorAttributes.put("error", "Internal Server Error");
        errorAttributes.put("message", "An unexpected error occurred");
        // Log lỗi chỉ tiết cho các lỗi không mong muốn
        System.err.println("Unhandled exception: " + ex.getMessage());
        ex.printStackTrace();
    }

    DataBufferFactory bufferFactory = response.bufferFactory();
    return
response.writeWith(Mono.just(bufferFactory.wrap(toJson(errorAttributes)).getB
});

private String toJson(Map<String, Object> map) {
    // Chuyển đổi Map sang JSON string (có thể dùng Jackson ObjectMapper)
    try {
        return new
com.fasterxml.jackson.databind.ObjectMapper().writeValueAsString(map);
    } catch (com.fasterxml.jackson.core.JsonProcessingException e) {
        return "{\"error\":\"Failed to serialize error response\"}";
    }
}
}

```

```

} ``

```

- **@ControllerAdvice / @RestControllerAdvice**: Vẫn có thể được sử dụng trong WebFlux, nhưng các phương thức **@ExceptionHandler** sẽ nhận các tham số reactive (Mono, Flux) và trả về các kiểu reactive.

```

`` `java package com.example.demo.exception;

```

```

import          org.springframework.http.HttpStatus;           import
org.springframework.http.ResponseEntity;                       import
org.springframework.web.bind.annotation.ExceptionHandler;      import

```

```
org.springframework.web.bind.annotation.RestControllerAdvice;      import
reactor.core.publisher.Mono;

import java.time.LocalDateTime;

@RestControllerAdvice public class ReactiveGlobalExceptionHandler {
```

```

    @ExceptionHandler(ResourceNotFoundException.class)
    public Mono<ResponseEntity<ErrorDetails>>
    handleResourceNotFoundException(ResourceNotFoundException ex) {
        ErrorDetails errorDetails = new ErrorDetails(
            LocalDateTime.now(), ex.getMessage(), "Resource Not Found");
        return Mono.just(new ResponseEntity<>(errorDetails,
            HttpStatus.NOT_FOUND));
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public Mono<ResponseEntity<ErrorDetails>>
    handleIllegalArgumentException(IllegalArgumentException ex) {
        ErrorDetails errorDetails = new ErrorDetails(
            LocalDateTime.now(), ex.getMessage(), "Invalid Argument");
        return Mono.just(new ResponseEntity<>(errorDetails,
            HttpStatus.BAD_REQUEST));
    }

    // Lớp ErrorDetails tương tự như ví dụ trước
    public static class ErrorDetails {
        private LocalDateTime timestamp;
        private String message;
        private Object details;

        public ErrorDetails(LocalDateTime timestamp, String message, Object
        details) {
            this.timestamp = timestamp;
            this.message = message;
            this.details = details;
        }

        // Getters and Setters
        public LocalDateTime getTimestamp() {
            return timestamp;
        }

        public void setTimestamp(LocalDateTime timestamp) {
            this.timestamp = timestamp;
        }

        public String getMessage() {
            return message;
        }

        public void setMessage(String message) {
            this.message = message;
        }

        public Object getDetails() {
            return details;
        }

        public void setDetails(Object details) {
            this.details = details;
        }
    }
}

```

```

} ``

```

---

## 7. Cấu hình và Tùy chỉnh Error Responses

Spring Boot cung cấp nhiều cách để cấu hình và tùy chỉnh phản hồi lỗi, đảm bảo rằng client nhận được thông tin lỗi nhất quán và dễ hiểu.

### 7.1. Cấu hình `application.properties` cho lỗi

Bạn có thể cấu hình một số thuộc tính liên quan đến lỗi trong `application.properties` (hoặc `application.yml`):

```
# Bật/tắt hiển thị thông báo lỗi chi tiết trong pha'n hoi
server.error.include-message=always

# Bật/tắt hiển thị stacktrace trong pha'n hoi
server.error.include-stacktrace=never # always, on_param, never

# Bật/tắt hiển thị binding errors (validation errors) trong pha'n hoi
server.error.include-binding-errors=always

# Đường dẫn mặc định cho trang lỗi (mặc định là /error)
server.error.path=/error

# Sử dụng Spring Boot's default error page (true) hay custom error page (false)
spring.mvc.throw-exception-if-no-handler-found=true # Ném
ExceptionHandler cho 404
spring.web.resources.add-mappings=false # Tắt mapping tài nguyên tĩnh để
ExceptionHandler hoạt động
```

### 7.2. Tùy chỉnh JSON Error Response

Khi sử dụng `@RestControllerAdvice`, bạn có toàn quyền kiểm soát cấu trúc JSON của phản hồi lỗi. Bạn nên tạo một lớp DTO (Data Transfer Object) để đại diện cho cấu trúc lỗi, giúp phản hồi nhất quán.

**Ví dụ về `ErrorResponse` DTO:**

```

package com.example.demo.dto;

import com.fasterxml.jackson.annotation.JsonFormat;
import com.fasterxml.jackson.annotation.JsonInclude;
import org.springframework.http.HttpStatus;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Map;

@JsonInclude(JsonInclude.Include.NON_NULL) // Không bao gồm các trường null
trong JSON
public class ErrorResponse {

    @JsonFormat(shape = JsonFormat.Shape.STRING, pattern = "yyyy-MM-dd
HH:mm:ss")
    private LocalDateTime timestamp;
    private int status;
    private String error;
    private String message;
    private String path;
    private Map<String, String> validationErrors; // Dành cho lỗi validation
    private List<String> details; // Dành cho các chi tiết lỗi khác

    public ErrorResponse() {
        this.timestamp = LocalDateTime.now();
    }

    public ErrorResponse(HttpStatus status, String message, String path) {
        this();
        this.status = status.value();
        this.error = status.getReasonPhrase();
        this.message = message;
        this.path = path;
    }

    // Getters and Setters
    public LocalDateTime getTimestamp() {
        return timestamp;
    }

    public void setTimestamp(LocalDateTime timestamp) {
        this.timestamp = timestamp;
    }

    public int getStatus() {
        return status;
    }

    public void setStatus(int status) {
        this.status = status;
    }

    public String getError() {
        return error;
    }

    public void setError(String error) {
        this.error = error;
    }
}

```

```
public String getMessage() {  
    return message;  
}  
  
public void setMessage(String message) {  
    this.message = message;  
}  
  
public String getPath() {  
    return path;  
}  
  
public void setPath(String path) {  
    this.path = path;  
}  
  
public Map<String, String> getValidationErrors() {  
    return validationErrors;  
}  
  
public void setValidationErrors(Map<String, String> validationErrors) {  
    this.validationErrors = validationErrors;  
}  
  
public List<String> getDetails() {  
    return details;  
}  
  
public void setDetails(List<String> details) {  
    this.details = details;  
}  
}
```

**Sử dụng ErrorResponse trong GlobalExceptionHandler :**

```

package com.example.demo.exception;

import com.example.demo.dto.ErrorResponse;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.HttpStatusCode;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.context.request.ServletWebRequest;
import org.springframework.web.context.request.WebRequest;
import org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler;

import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler extends ResponseEntityExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse>
handleResourceNotFoundException(ResourceNotFoundException ex, WebRequest
request) {
        ErrorResponse errorResponse = new ErrorResponse(
            HttpStatus.NOT_FOUND, ex.getMessage(), ((ServletWebRequest)
request).getRequest().getRequestURI());
        return new ResponseEntity<>(errorResponse, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(DuplicateProductException.class)
    public ResponseEntity<ErrorResponse>
handleDuplicateProductException(DuplicateProductException ex, WebRequest
request) {
        ErrorResponse errorResponse = new ErrorResponse(
            HttpStatus.CONFLICT, ex.getMessage(), ((ServletWebRequest)
request).getRequest().getRequestURI());
        return new ResponseEntity<>(errorResponse, HttpStatus.CONFLICT);
    }

    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(
        MethodArgumentNotValidException ex, HttpHeaders headers,
        HttpStatusCode status, WebRequest request) {

        Map<String, String> validationErrors = new HashMap<>();
        ex.getBindingResult().getAllErrors().forEach(error -> {
            String fieldName = ((FieldError) error).getField();
            String errorMessage = error.getDefaultMessage();
            validationErrors.put(fieldName, errorMessage);
        });

        ErrorResponse errorResponse = new ErrorResponse(
            HttpStatus.BAD_REQUEST, "Validation failed",
            ((ServletWebRequest) request).getRequest().getRequestURI());
        errorResponse.setValidationErrors(validationErrors);

        return new ResponseEntity<>(errorResponse, HttpStatus.BAD_REQUEST);
    }
}

```

```

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleAllUncaughtException(Exception
ex, WebRequest request) {
        // Log lỗi chi tiết ở đây
        System.err.println("An unexpected error occurred: " + ex.getMessage());
        ex.printStackTrace();

        ErrorResponse errorResponse = new ErrorResponse(
            HttpStatus.INTERNAL_SERVER_ERROR, "An unexpected error
occurred", ((ServletWebRequest) request).getRequest().getRequestURI());
        return new ResponseEntity<>(errorResponse,
            HttpStatus.INTERNAL_SERVER_ERROR);
    }
}

```

### 7.3. Sử dụng `ProblemDetail` (Spring 6+)

Spring Framework 6 (và Spring Boot 3) giới thiệu `ProblemDetail` như một cách tiêu chuẩn hóa để trả về phản hồi lỗi theo RFC 7807 (Problem Details for HTTP APIs). Điều này giúp client dễ dàng hiểu và xử lý các lỗi API một cách nhất quán.

#### Cách sử dụng:

- `ProblemDetail` là một lớp mới trong Spring Framework.
- Bạn có thể trả về `ProblemDetail` từ các phương thức `@ExceptionHandler`.

#### Ví dụ:



```

package com.example.demo.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ProblemDetail;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.net.URI;
import java.time.Instant;

@RestControllerAdvice
public class ProblemDetailExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ProblemDetail
    handleResourceNotFoundException(ResourceNotFoundException ex) {
        ProblemDetail problemDetail =
        ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
        problemDetail.setTitle("Resource Not Found");
        problemDetail.setType(URI.create("/errors/not-found"));
        problemDetail.setProperty("timestamp", Instant.now());
        return problemDetail;
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ProblemDetail
    handleIllegalArgumentException(IllegalArgumentException ex) {
        ProblemDetail problemDetail =
        ProblemDetail.forStatusAndDetail(HttpStatus.BAD_REQUEST, ex.getMessage());
        problemDetail.setTitle("Invalid Argument");
        problemDetail.setType(URI.create("/errors/invalid-argument"));
        problemDetail.setProperty("timestamp", Instant.now());
        return problemDetail;
    }

    @ExceptionHandler(Exception.class)
    public ProblemDetail handleGenericException(Exception ex) {
        ProblemDetail problemDetail =
        ProblemDetail.forStatusAndDetail(HttpStatus.INTERNAL_SERVER_ERROR, "An
        unexpected error occurred");
        problemDetail.setTitle("Internal Server Error");
        problemDetail.setType(URI.create("/errors/internal-server-error"));
        problemDetail.setProperty("timestamp", Instant.now());
        // Có thể thêm stacktrace vào đây trong môi trường dev/test
        return problemDetail;
    }
}

```

Phản hồi JSON sẽ tuân thủ RFC 7807:

```

{
  "type": "/errors/not-found",
  "title": "Resource Not Found",
  "status": 404,
  "detail": "Product not found with ID: 123",
  "instance": "/api/products/123",
  "timestamp": "2023-10-27T10:30:00Z"
}

```

---

## 8. Best Practices và Lời khuyên

---

### 8.1. Không nuốt ngoại lệ (Don't swallow exceptions)

Tránh bắt ngoại lệ mà không xử lý hoặc ghi log chúng. Việc "nuốt" ngoại lệ (empty `catch` block) sẽ khiến việc debug trở nên cực kỳ khó khăn.

```
// KHÔNG NÊN LÀM
try {
    // ... code ...
} catch (Exception e) {
    // Không làm gì cả
}

// NÊN LÀM
try {
    // ... code ...
} catch (Exception e) {
    logger.error("An error occurred: ", e); // Luôn ghi log
    // Hoặc ném lại một ngoại lệ khác có ý nghĩa hơn
    throw new CustomBusinessException("Failed to process", e);
}
```

### 8.2. Log ngoại lệ đúng cách

Luôn ghi log ngoại lệ với mức độ phù hợp ( `ERROR` , `WARN` ) và bao gồm stacktrace để dễ dàng debug. Sử dụng logger của SLF4J/Logback.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

private static final Logger logger = LoggerFactory.getLogger(MyClass.class);

// ...

try {
    // ...
} catch (Exception e) {
    logger.error("Error processing request for user {}", userId, e); // Truyền
    // thông tin ngoại lệ
}
```

### 8.3. Tạo thông báo lỗi thân thiện với người dùng

Phản hồi lỗi cho client nên rõ ràng, dễ hiểu và không tiết lộ thông tin nhạy cảm (ví dụ: chi tiết database, stacktrace đầy đủ trong môi trường production).

```
// Tốt
{
  "status": 404,
  "message": "Sản phẩm không tìm thấy với ID: 123"
}

// Không tốt (tiết lộ thông tin nội bộ)
{
  "timestamp": "2023-10-27T10:30:00.123Z",
  "status": 500,
  "error": "Internal Server Error",
  "trace": "java.sql.SQLException: Connection refused: connect\n\tat
com.mysql.jdbc.Connection.connect(Connection.java:2345)..."
}
```

### 8.4. Phân biệt lỗi nghiệp vụ và lỗi hệ thống

- **Lỗi nghiệp vụ (Business Exceptions):** Các lỗi do vi phạm quy tắc nghiệp vụ (ví dụ: số dư không đủ, sản phẩm hết hàng). Thường là `RuntimeException` tùy chỉnh, được ánh xạ tới HTTP 4xx (Bad Request, Conflict, Not Found).
- **Lỗi hệ thống (System Exceptions):** Các lỗi liên quan đến hạ tầng, kết nối, code (ví dụ: `NullPointerException`, `IOException`, lỗi database). Thường là `RuntimeException` hoặc `Error`, được ánh xạ tới HTTP 5xx (Internal Server Error, Service Unavailable).

### 8.5. Sử dụng mã lỗi (Error Codes)

Đối với các ứng dụng lớn, việc sử dụng mã lỗi (error codes) giúp client dễ dàng xử lý các loại lỗi cụ thể mà không cần phân tích thông báo lỗi. Bạn có thể thêm trường `code` vào `ErrorResponse` DTO của mình.

```

public enum ErrorCode {
    PRODUCT_NOT_FOUND("P001", "Product not found"),
    INSUFFICIENT_STOCK("S002", "Insufficient stock"),
    INVALID_INPUT("V003", "Invalid input data"),
    // ...

    private final String code;
    private final String message;

    ErrorCode(String code, String message) {
        this.code = code;
        this.message = message;
    }

    public String getCode() {
        return code;
    }

    public String getMessage() {
        return message;
    }
}

// Trong ErrorResponse
public class ErrorResponse {
    // ...
    private String code;
    // ...
}

// Trong GlobalExceptionHandler
@ExceptionHandler(ResourceNotFoundException.class)
public ResponseEntity<ErrorResponse>
handleResourceNotFoundException(ResourceNotFoundException ex, WebRequest
request) {
    ErrorResponse errorResponse = new ErrorResponse(
        HttpStatus.NOT_FOUND, ex.getMessage(), ((ServletWebRequest)
request).getRequest().getRequestURI());
    errorResponse.setCode(ErrorCode.PRODUCT_NOT_FOUND.getCode());
    return new ResponseEntity<>(errorResponse, HttpStatus.NOT_FOUND);
}

```

## 8.6. Kiểm thử xử lý ngoại lệ

Luôn viết kiểm thử cho các kịch bản lỗi để đảm bảo rằng ứng dụng xử lý ngoại lệ đúng cách và trả về phản hồi lỗi mong muốn.

```

@WebMvcTest(ProductController.class)
public class ProductControllerExceptionTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private ProductService productService;

    @Test
    void whenProductNotFound_thenReturns404() throws Exception {
        when(productService.getProductById(anyLong()))
            .thenReturn(Optional.empty());

        mockMvc.perform(get("/api/products/999"))
            .andExpect(status().isNotFound())
            .andExpect(jsonPath("$.status").value(404))
            .andExpect(jsonPath("$.message").value("Product not found with
ID: 999"));
    }

    @Test
    void whenInvalidInput_thenReturns400() throws Exception {
        mockMvc.perform(post("/api/products")
            .contentType(MediaType.APPLICATION_JSON)
            .content("{\"name\":\"a\", \"price\":-1}"))
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.status").value(400))
            .andExpect(jsonPath("$.message").value("Validation failed"))
            .andExpect(jsonPath("$.validationErrors.name").exists())
            .andExpect(jsonPath("$.validationErrors.price").exists());
    }
}

```

## Kết luận

Xử lý ngoại lệ là một phần không thể thiếu trong việc phát triển các ứng dụng Spring Boot chất lượng cao. Bằng cách áp dụng các cơ chế do Spring cung cấp như `@ControllerAdvice`, `@ExceptionHandler`, và `ProblemDetail`, cùng với việc tạo các ngoại lệ tùy chỉnh và tuân thủ các best practices, bạn có thể xây dựng các ứng dụng mạnh mẽ, dễ bảo trì và cung cấp trải nghiệm người dùng tốt hơn ngay cả khi có lỗi xảy ra.

## Tài liệu tham khảo

[1] Baeldung - Exception Handling for REST with Spring: <https://www.baeldung.com/exception-handling-for-rest-with-spring> [2] Spring Boot Official Documentation - Error Handling: <https://docs.spring.io/spring->

[boot/docs/current/reference/html/web.html#web.servlet.error-handling](https://docs.spring.io/spring-framework/docs/current/reference/html/web.html#web.servlet.error-handling) [3]  
GeeksforGeeks - Exception Handling in Spring Boot:  
<https://www.geeksforgeeks.org/springboot/exception-handling-in-spring-boot/> [4]  
Spring Framework Official Documentation - Async Support:  
<https://docs.spring.io/spring-framework/docs/current/reference/html/integration.html#scheduling-async-exceptions> [5] Spring Framework Official Documentation - WebFlux Error Handling:  
<https://docs.spring.io/spring-framework/docs/current/reference/html/web-reactive.html#webflux-exception-handling> [6] RFC 7807 - Problem Details for HTTP APIs: <https://tools.ietf.org/html/rfc7807>